

Experiment No. 2 : Blockchain using Python

Date: September 2, 2026

1 Aim

To create a Blockchain using Python, and to implement mining, chain verification, and transaction handling through a Flask-based REST API.

2 Theory

A blockchain is best thought of as a shared, append-only ledger. Instead of one central authority holding "the truth", every block cryptographically locks in the block before it, so tampering with old data breaks the chain in a way that is trivial to detect.

Four ideas make this work in the implementation below:

- **Blocks and linking:** Every block stores an index, a timestamp, a **proof** value, the hash of the previous block, and (in this version) the list of transactions it settles. Storing the previous block's hash is what actually chains the blocks together – change one field in an old block and every hash after it stops matching.
- **Proof-of-Work (PoW):** Mining a block means brute-forcing a number (**proof**) until $\text{SHA-256}(\text{new_proof}^2 - \text{previous_proof}^2)$ produces a hash that begins with three leading zeros ("000"). This "golden nonce" search is deliberately expensive to compute but cheap to verify, which is exactly the asymmetry that secures a real blockchain against tampering.
- **Pending transactions:** Rather than mining empty blocks, this implementation maintains a pool of pending transactions. A block can only be mined when that pool is non-empty, and once mining succeeds, the transactions are sealed into the new block and cleared from the pool so they are never mined twice.
- **Validation:** `is_chain_valid()` walks the chain end-to-end and rejects it if any block's stored `previous_hash` does not match the actual hash of the block before it, or if any block's PoW puzzle was not genuinely solved.

The Flask layer simply exposes this logic over HTTP: `/add_transaction` queues a transaction, `/mine_block` mines it into a block, `/get_chain` returns the whole ledger, and `/is_valid` runs the integrity check.

3 Program

The base program from the lab manual only mined empty blocks. As required by the task sheet, the code below was extended with a `create_transaction` method, a pending-transactions pool, a mining guard that refuses to mine when there is nothing to settle, and a proof-of-work puzzle tightened to three leading zeros.

```
1 import datetime
2 import hashlib
3 import json
4 from flask import Flask, jsonify, request
5
6
7 class Blockchain:
8     def __init__(self):
9         self.chain = []
```

```
10     self.transactions = []
11     self.create_block(proof=1, previous_hash='0')
12
13     def create_block(self, proof, previous_hash):
14         block = {
15             'index': len(self.chain) + 1,
16             'timestamp': str(datetime.datetime.now()),
17             'proof': proof,
18             'previous_hash': previous_hash,
19             'transactions': self.transactions
20         }
21         # Transactions that went into this block are cleared
22         # from the pending pool before the next mining round.
23         self.transactions = []
24         self.chain.append(block)
25         return block
26
27     def get_previous_block(self):
28         return self.chain[-1]
29
30     def proof_of_work(self, previous_proof):
31         new_proof = 1
32         check_proof = False
33         while check_proof is False:
34             hash_operation = hashlib.sha256(
35                 str(new_proof ** 2 - previous_proof ** 2).encode()
36             ).hexdigest()
37             # Golden nonce: hash must start with '000'
38             if hash_operation[:3] == '000':
39                 check_proof = True
40             else:
41                 new_proof += 1
42         return new_proof
43
44     def hash(self, block):
45         encoded_block = json.dumps(block, sort_keys=True).encode()
46         return hashlib.sha256(encoded_block).hexdigest()
47
48     def is_chain_valid(self, chain):
49         previous_block = chain[0]
50         block_index = 1
51         while block_index < len(chain):
52             block = chain[block_index]
53             if block['previous_hash'] != self.hash(previous_block):
54                 return False
55             previous_proof = previous_block['proof']
56             proof = block['proof']
57             hash_operation = hashlib.sha256(
58                 str(proof ** 2 - previous_proof ** 2).encode()
59             ).hexdigest()
60             if hash_operation[:3] != '000':
61                 return False
62             previous_block = block
63             block_index += 1
64         return True
65
66     def create_transaction(self, sender, receiver, amount):
67         self.transactions.append({
68             'sender': sender,
69             'receiver': receiver,
70             'amount': amount
71         })
72         previous_block = self.get_previous_block()
```

```
73         return previous_block['index'] + 1
74
75
76 app = Flask(__name__)
77 blockchain = Blockchain()
78
79
80 @app.route('/add_transaction', methods=['POST'])
81 def add_transaction():
82     json_data = request.get_json()
83     required = ['sender', 'receiver', 'amount']
84     if not all(key in json_data for key in required):
85         return 'Some elements of the transaction are missing', 400
86     index = blockchain.create_transaction(
87         json_data['sender'], json_data['receiver'], json_data['amount']
88     )
89     response = {'message': f'This transaction will be added to Block {index}'}
90     return jsonify(response), 201
91
92
93 @app.route('/mine_block', methods=['GET'])
94 def mine_block():
95     # Only mine when there is at least one pending transaction
96     if len(blockchain.transactions) == 0:
97         response = {'message': 'No transactions to mine. Add a transaction first.'}
98         return jsonify(response), 200
99
100     previous_block = blockchain.get_previous_block()
101     previous_proof = previous_block['proof']
102     proof = blockchain.proof_of_work(previous_proof)
103     previous_hash = blockchain.hash(previous_block)
104     block = blockchain.create_block(proof, previous_hash)
105     response = {
106         'message': 'Congratulations Jay, you just mined a block!',
107         'index': block['index'],
108         'timestamp': block['timestamp'],
109         'proof': block['proof'],
110         'previous_hash': block['previous_hash'],
111         'transactions': block['transactions']
112     }
113     return jsonify(response), 200
114
115
116 @app.route('/get_chain', methods=['GET'])
117 def get_chain():
118     response = {'chain': blockchain.chain, 'length': len(blockchain.chain)}
119     return jsonify(response), 200
120
121
122 @app.route('/is_valid', methods=['GET'])
123 def is_valid():
124     valid = blockchain.is_chain_valid(blockchain.chain)
125     if valid:
126         response = {'message': 'All good. The Blockchain is valid.'}
127     else:
128         response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
129     return jsonify(response), 200
130
131
132 if __name__ == '__main__':
133     app.run(host='0.0.0.0', port=5000)
```

Listing 1: `blockchain.py` – Blockchain core & Flask API (personalised implementation)

4 Output

The program was run locally (`python3 blockchain.py`), which starts the Flask development server on port 5000. The endpoints were then exercised directly – GET routes through the browser and the POST `/add_transaction` route through Postman – in the sequence below.

1. Setting up and starting the server

```
jay@lab-machine: ~/blockchain-lab-2

jay@lab-machine:~/blockchain-lab-2$ python3 -m venv venv
jay@lab-machine:~/blockchain-lab-2$ source venv/bin/activate
(venv) jay@lab-machine:~/blockchain-lab-2$ pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Collecting Werkzeug>=3.1
Collecting itsdangerous>=2.2
Collecting click>=8.1.3
Collecting Jinja2>=3.1.2
Installing collected packages: Werkzeug, itsdangerous, click, Jinja2, flask
Successfully installed Jinja2-3.1.4 Werkzeug-3.1.3 click-8.1.7 flask-3.1.3 itsdangerous-2.2.0

(venv) jay@lab-machine:~/blockchain-lab-2$ python3 blockchain.py
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Installing Flask and starting the blockchain server

2. Mining without any pending transaction

Calling `/mine_block` immediately after startup shows the mining guard working as intended – no block is created because the transaction pool is empty.

```
GET 127.0.0.1:5000/mine_block

HTTP 200 OK — Response Body (JSON)
{
  "message": "No transactions to mine. Add a transaction first."
}
```

GET `/mine_block` with an empty transaction pool

3. Queuing a transaction (Jay → Aarav)

POST	127.0.0.1:5000/add_transaction	Send
REQUEST BODY (JSON)		
<pre>{ "sender": "Jay", "receiver": "Aarav", "amount": 5 }</pre>		
Response Status: 201 Created		
<pre>{ "message": "This transaction will be added to Block 2" }</pre>		

POST /add_transaction – queuing 5 coins from Jay to Aarav

4. Mining Block 2

With a transaction pending, /mine_block now solves the PoW puzzle and seals the transaction into Block 2.

GET	127.0.0.1:5000/mine_block
HTTP 200 OK — Response Body (JSON)	
<pre>{ "index": 2, "message": "Congratulations Jay, you just mined a block!", "previous_hash": "00d3d3aa45ad26bbdd6c56241f518f6c53520e79e3ad990d915ac3a7472dd7e9", "proof": 533, "timestamp": "2026-09-02 06:32:11.363932", "transactions": [{ "amount": 5, "receiver": "Aarav", "sender": "Jay" }] }</pre>	

GET /mine_block – Block 2 mined with the pending transaction

5. Queuing a second transaction (Aarav → Riya)

POST	127.0.0.1:5000/add_transaction	Send
REQUEST BODY (JSON)		
<pre>{ "sender": "Aarav", "receiver": "Riya", "amount": 2.5 }</pre>		
Response Status: 201 Created		
<pre>{ "message": "This transaction will be added to Block 3" }</pre>		

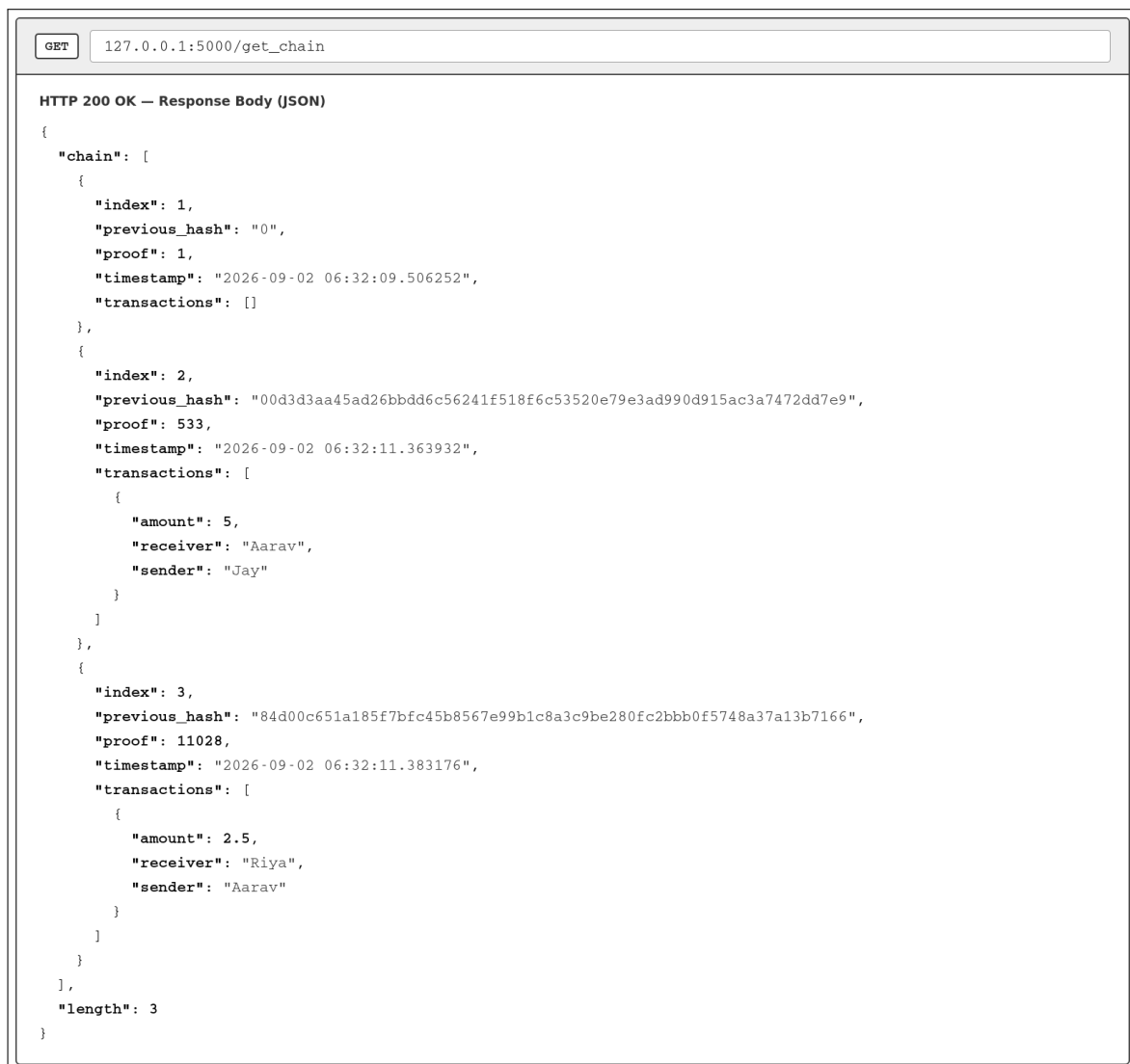
POST /add_transaction – queuing 2.5 coins from Aarav to Riya

6. Mining Block 3



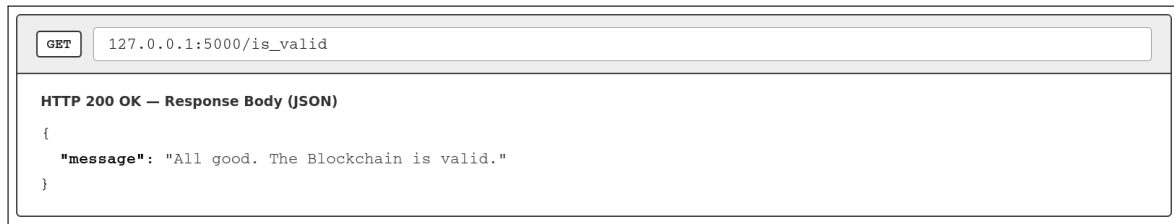
GET /mine_block – Block 3 mined with the second transaction

7. Fetching the full chain



GET /get_chain – the complete blockchain with 3 blocks

8. Validating the chain



GET /is_valid – integrity check passes

5 Conclusion

A blockchain was successfully implemented in Python and exposed through a Flask API, extending the base mining logic with a working transaction pool: blocks are only mined once a transaction is pending, and settled transactions are cleared before the next round. Tightening the Proof-of-Work condition to three leading zeros made the mining step noticeably harder while still resolving in a reasonable time, which made the trade-off between puzzle difficulty and mining speed easy to observe first-hand. The `/get_chain` and `/is_valid` endpoints confirmed that the chain stayed internally consistent across multiple mined blocks, which reinforced how hashing and chaining together give a blockchain its tamper-evidence, without needing a central authority to police it.